

太原理工大学先进计算机系统实验室（ACSL）

正式学员第一次学习路线

学习情况：寒假研学结营，完成数据结构的学习，持续进行着Verilog实践练习和LCTHW实践。

学习目标：本周为开学第一周，经过一寒假的研学，相信大家都有些许疲惫，但依照往年经验，一旦学习的势头断了之后就会疯狂摆烂，因此开学之后的学习任务也是不会停滞的，本周依旧是Verilog实践和LCTHW实践练习。

数字设计

Verilog

本周你需要完成HDLBits中的如下任务，提交你的截图即可。（在这里推荐一个浏览器插件：**沉浸式翻译**，如果看不懂的话，就用这个插件配合学习吧！）

新推荐

如果你更喜欢更原汁原味地浏览英文网页，这里有个更好的选择：[tapword-translator\(www.tapword.cc\)](https://www.tapword.cc)。

如果你也有推荐的其他工具，欢迎在群内多多交流。

- Counters
 - Decade counter again
 - Slow decade counter
 - Counter 1000
 - 12-hour clock
- Shift Registers
 - Left/right rotator
 - Left/right arithmetic shift by 1 or 8
 - 5-bits LFSR
 - Shift register
 - 3-input LUT
- More Circuits - 全部完成

！ 我们培养的是硬件思维，需要头脑中先有电路再下手写代码，这也是为什么我们需要先学习使用Logisim搭建数字电路，再来学习数字设计，虽然我们后面不再使用Logisim进行处理器设计，但Logisim的使用经验应该已经帮助你建立了"电路思维"：数字电路设计只做两件事，"实例化"和"连线"。你接下来使用HDL来设计数字电路时，头脑中也需要将HDL代码和Logisim的使用经验建立关联：你只不过是换了一种方式来设计电路，但本质上还是在进行"实例化"和"连线"的工作，因此你应该能根据你编写的代码想象到电路的逻辑结构，**要记住Verilog的本质是硬件描述语言而不是传统的编程语言。**

Learn C the hard way

函数指针

本周的Lcthw第18篇中你们会接触一个新的概念：函数指针。

我们学过指针，学过结构体指针，那函数指针是什么东西，都学习这么久了，你肯定能理解函数指针不就是指向函数的指针吗，相应的，它存储的就是函数的地址，原理很简单，那它到底有什么用呢？

为什么要有函数指针？（最常见的用途）

其实在C语言中，**函数名本身就是一个指向函数的指针**。当我们在代码中使用函数名时（不带括号），实际上我们使用的是函数的地址。那拿到地址可以干什么呢？

函数的地址相当于是函数的入口，函数指针指向它，则说明我们可以通过不断改变入口来切换函数指针的所指向的函数，简单说：函数指针就像一个“遥控器”，可以随时切换控制不同的函数。它能让我们把**“不同的行为”当作参数传给别人**。类比一下就是：

- 实现排序的函数中可以传不同的“比较规则”。同一个 sort 函数，你可以让它按从小到大排数字，也可以按从大到小排，甚至按字符串长度排……只要你把不同的“比较方式”（函数）作为参数传进去就行。这也正是本章 Lcthw 所实现的。

函数指针的写法看起来有点怪：返回类型 (*指针名字)(参数列表)：

<code>void (*function)(void)</code>	指向“无参数、无返回值的函数”的指针
<code>int (*function)(int, int)</code>	指向“两个 int 参数、返回 int”的函数
<code>void (*function)(int)</code>	指向“一个 int 参数、没返回值”的函数

而函数指针非常灵活，它允许我们将函数作为参数传递给其他函数，或者将函数作为返回值从函数中返回。以下为2种示例：

代码块

```
1  int add(int a, int b) {return a + b;}
2  int subtract(int a, int b) {return a - b;}
3
4  int main() {
5      int (*operation)(int, int); // 声明一个函数指针
6
7      operation = add; // 将函数指针指向 add 函数
8      printf("加法结果: %d\n", operation(5, 3));
9
10     operation = subtract; // 将函数指针指向 subtract 函数
11     printf("减法结果: %d\n", operation(5, 3));
12     return 0;
13 }
```

代码块

```
1  void terrible_err(void){
2      printf(format:"Error:Meet a severe Error\n");
3  }
4
5  void bad_err(void){
6      printf(format:"Error: Meet a bad ErrorIn");
7  }
8
9  typedef void (*err)(void);
10
11 void print_err(err e){
12     e();
13 }
```

```
13 }
14
15 int main(){
16     print_err(terrible_err);
17     print_err(bad_err);
18 }
```

函数指针还有一个经典的作用就是用来实现回调函数，这又是一个新的概念，那么回调函数又是什么呢？

回调函数

回调函数 (callback)，一个很生动形象的理解就是“**你先告诉我会要做什么，我到时候再叫你**”。

稍微专业一点的说法就是你把一个函数的地址（函数指针）传给系统/函数，告诉它：“将来某个时刻你遇到这种情况，就帮我调用这个函数” → 这就叫**回调**。

我们推荐以下的博客来帮助你更好的理解它（写的比较详细，有图有示例，后面有关线程的部分可以先不看）：

<https://zhuanlan.zhihu.com/p/640735987>

? 烧烤一下

那 回调函数 与 普通的函数调用 有什么区别？

以上是简单的函数指针入门，函数指针作为C语言中非常灵活，非常强大的工具之一，你可以试着自己多去写几个例子（乘除法切换，自定义排序等），很快你就会觉得“哇，还能这么玩！”

? 提问：你分的清 指针函数 和 函数指针 吗

如果还有不清楚的，请积极STFW。

如果你对于函数的运行感兴趣，可以观看该视频间接了解：[CPU如何运行函数？](#)

调试宏

Lcthw的第20篇部分的内容涉及到一个较难的概念——调试宏。

刚开始接触这个概念你可能会很模糊，并且可能很久都搞不懂它有什么作用，但是深入了解之后你会发现相关的功能在调试中还是很好用的。但在此之前，你知道宏是什么吗？

宏

为了搞懂什么是调试宏，首先我们应该先对宏有个清晰的认识，我们接下来先来定义一个简单的宏：

代码块

```
1  // 以下是两种简单的宏定义
2  #define PI_1 3.14          // 对象宏
3  #define S(a,b) a*b         // 函数宏
4
5  #include <stdio.h>
6
7  int main()
8  {
9      printf("%d", PI_1);
10     printf("%d * %d = %d", S(1,2) , 3 , S(1,2) * 3 );
11     return 0;
12 }
```

如果你已经读懂了以上的代码，那恭喜你已经初步认识到了宏的定义方法以及两种基础分类，如果你仍有些许迷茫，那建议你去好好复习一下C语言的相关知识，那接下来我们就来深入了解调试宏的相关内容。

LCTHW中的调试宏依赖于一个完整的头文件 `dbg.h` ,你可以看到在这个头文件中定义了许多非常复杂的宏，他们就是适用于调试的所有宏。在正式开始进行调试宏的学习前你还需要另外两个基础知识的学习。

? STFSC + man

查看 `errno.h` （是标准C库的一部分）的相关内容，重点理解`errno`宏的功能，如果你还不知道如何使用命令行查看的话，那也请你继续STFW复习一下相关的内容。

通过使用 `man stderr` 命令查看 `stderr` 和我们之前熟悉的 `stdout` 有什么区别。

在有了上述前提知识的储备知识之后，我们就正式开始相关内容的学习，现在来根据调试宏相关的功能将 `dbg.h` 文件中的宏进行区分。（这一步你通过查看每个宏定义的名称就可以完成，这也是很好的函数定义命名的例子）

- 调试宏定义
- 清理错误宏
- 日志宏
- 检查宏

在完整区分相关的内容之后请你对代码的相应地方进行注释，这是为了方便之后对相关函数进行调用时可以更方便的对照。

? STFW + 思考

在 `dbg.h` 文件中你会发现 `##VA_ARGS` 的一个预处理运算符+特殊标识符的结构，请你通过**STFW**，来查找它究竟代表什么，并且思考它在 `dbg.h` 中的作用是什么？为了检测你是否真正理解了相关内容请你尝试阐述在以下的代码中 `##VA_ARGS` 的作用：

代码块

```
1 // 以下是 logger.h 头文件的相关内容
2 #ifndef LOGGER_H
3 #define LOGGER_H
4
5 #include <stdio.h>
6
7 #ifdef DEBUG
8 #define LOG_DEBUG(format, ...) \
9     fprintf(stderr, "DEBUG: %s:%d: " format "\n", __FILE__, __LINE__,
10     ##__VA_ARGS__)
11 #else
12 #define LOG_DEBUG(format, ...) // 空定义
13 #endif
14 #endif // LOGGER_H
```

代码块

```
1 #include "logger.h"
2
3 int main() {
4     int a = 5;
5     int b = 10;
6
7     LOG_DEBUG("Values: a = %d, b = %d", a, b);
8     LOG_DEBUG("This is a debug message without extra arguments");
9
10    return 0;
11 }
```

? 在以上的代码中还展示了 `##VA_ARGS` 的一个重要特性，你发现了吗？

在读懂了上述示例代码的内容之后，你就应该可以读懂LCTHW中使用**dbg.h**部分的内容了。

以上便是调试宏在实际中的运用了，它可以做到许多我们一般使用的printf（探针），做不到的调试效果：

- a. 可以及时控制调试与否，通过#ifdef控制DEBUG还是NDEBUG就可以实现这个简单的功能；
- b. 可以利用预定义宏（如 `FILE`、`LINE`、`func`）自动输出文件名、行号、函数名，同时也支持可变参数；
- c. 可以统一调试输出，更加规整；
- d. 可以通过定义的调试宏实现不同级别的过滤（ERROR、INFO、DEBUG等）；

以上都是一些调试宏可以做到，但是printf得经过其他手段才可以做到的事，这就是我们要学习使用调试宏的原因，但是在有以上便利的同时它确实也存在许多的缺点：

- a. 具有更强的语法复杂性，要是不熟悉相关的语法经常会出现输出不符合预期的情况；
- b. 宏是基于文本替换的，如果传递给调试宏的参数包含 `++x` 或者函数调用，可能会被多次求值，导致非预期行为；
- c. 难以调试，因为宏展开的缘故，会导致在使用gdb等调试器单步执行时指示向展开后的代码。

因为存在这以上的优缺点，所以调试宏只作为一个拓展性的工具来让大家了解其运行方式，感兴趣的同学日后在写代码时可以使用相关的方法来让自己的代码更加健壮。

Learn C the hard way

可能有的同学已经忘记了，我们的C语言程序是要在**Linux环境**上写的，本周完成其中的**第18，20章**

【Learn C the hard way】：

<https://wizardforcel.gitbooks.io/lcthw/content/preface.html>

将你完成的所有练习放入一个名为 `lcthw` 的文件夹，并将该文件夹放入作业提交文件夹中。

! 如果你已经做完，可以选择复习一遍，或移步拔高。

作业提交



1. HDLBits的截图命名为 `Verilog` 并放到 `姓名-专业班级-Great-14` 文件夹中。
2. `lcthw` 文件夹也放在上述文件夹里。

3. 如果你学有余力完成了下面的拔高内容，则把文件夹重命名，格式为 姓名-专业班级-NewStar-14。
4. 将你的作业压缩为zip格式并提交到[作业提交表单](#)。

拔高内容

Learn C the hard way

这是“一生一芯”的**必须完成部分**如下：

Linux中的C语言进阶学习

在Linux中完成[Learn C the hard way](#)练习0~22, 练习24~25, 练习27~33, 你需要将示例代码拷贝到Linux中编译并运行, 结合文字RTFSC理解示例代码, 并完成相应的附加题。

注意, 上述习题将是入学答辩的一部分. 如果你被某些题目卡住, 可以通过STFW或在预学习群中询问.

虽然这份教程的C语言知识点和之前的《Linux C编程一站式学习》差不多, 但它还包含了Makefile, Valgrind等各种工具的使用介绍, 以及一些值得学习的编程理念, 这些对大家将来进行开发都是很有帮助的, 非常值得大家在Linux上进行实践. 事实上, 如果你之前的C语言基础足够牢固, 你不应该在阅读代码上花费过多的时间; 相反, 如果你在阅读代码时遇到不少困难, 你应该感谢这次机会帮助你查缺补漏.

虽然一生一芯的讲义划定了学习的范围，但想要技术很强的话，我们建议都可以试着去学习。

Learn C the hard way

其中的**26、37-41、43、45-47不需要学习**，性价比比较低，不推荐学习，**其他内容我们都很推荐学习**，想要技术很强的话，都可以试着去学习，并在其中锻炼自己gdb等debug工具使用和相关能力思维。

【Learn C the hard way】：<https://wizardforcel.gitbooks.io/lcthw/content/preface.html>

将你完成的所有练习放入一个名为 `lcthw` 的文件夹，并将该文件夹放入作业提交文件夹中。

一生一芯课程PA

PA是我们后续学习中非常重要的一部分内容，目前我们已经把PA0相关的基础知识进行了补全，大家**可以去尝试PA0的相关内容学习**，而后面的PA1是一生一芯预学习答辩的主要内容之一，如果你想挑战自己，现在去尝试PA1也是没问题的。

! 当你发现如下提醒时，阅读该讲义:<https://ysyx.oscc.cc/docs/2407/e/3.html>获取属于一生一芯的代码框架

! 参加"一生一芯"的同学, 请参考"一生一芯"讲义获取代码链接

如果你参加"一生一芯", 请勿使用下面的代码链接. 此外, PA讲义中关于作业提交的要求, "一生一芯"的同学可以全部忽略, 但需要关注"一生一芯"讲义中的提交要求.